# Rechnersysteme – Befehlsübersicht Praxis

Raspberry Pi · Linux/Bash · Python · GPIO · Sensoren · Netzwerk

DHBW Karlsruhe – Dozent Dipl.-Ing. Dirk Hotz

## Teil 1 – Linux / Bash

### Grundlegende Shell-Bedienung

| Aktion | Beschreibung |
|---|---|
| `Cursor-Up/Down` | Vorherige Befehle wiederholen |
| `Tab` | Automatische Ergänzung von Dateinamen |
| `Ctrl-a / Ctrl-e` | Anfang / Ende der Zeile |
| `Ctrl-z` | Laufenden Prozess stoppen |
| `Ctrl-c` | Laufenden Prozess abbrechen |

### Dateisystem-Befehle

| Befehl | Beschreibung |
|---|---|
| `man <prog>` | Hilfe anzeigen (verlassen mit `q`) |
| `ls <file>...` | Dateien auflisten |
| `ls -l <file>` | Dateirechte und Details ausgeben |
| `ls -a` | Auch versteckte Dateien anzeigen |
| `cp [-r] <src> <dst>` | Dateien kopieren (`-r`: rekursiv) |
| `mv <src> <dst>` | Dateien verschieben / umbenennen |
| `rm [-r] <file>` | Dateien löschen |
| `pwd` | Aktuelles Verzeichnis ausgeben |
| `mkdir <dir>` | Verzeichnis erzeugen |
| `mkdir -p <dir>` | Verzeichnis inkl. Elternverzeichnisse erzeugen |
| `cd <dir>` | Arbeitsverzeichnis wechseln |
| `touch <datei>` | Leere Datei erzeugen |

#### Besondere Pfade

| | |
|---|---|
| `.` | Aktuelles Verzeichnis |
| `..` | Übergeordnetes Elternverzeichnis |
| `~` | Eigenes Home-Verzeichnis |
| `~name` | Home-Verzeichnis des Benutzers `name` |
| `/` | Wurzelverzeichnis (root) |

### Wichtige Verzeichnisse

| | |
|---|---|
| `/bin` | Grundlegende Programme (ls, cat, cp, ...) |
| `/boot` | Bootloader inkl. Kernel |
| `/dev` | Gerätedateien (`/dev/sda1`, `/dev/null`, `/dev/tty`) |
| `/etc` | Systemkonfiguration |
| `/home` | Benutzerverzeichnisse |
| `/lib` | Bibliotheken |
| `/media` | Temporär eingehängte Medien (CD, USB) |
| `/mnt` | Temporär eingehängte Dateisysteme |
| `/opt` | Zusätzliche Anwendungsprogramme |
| `/proc` | Systeminformationen |
| `/root` | Home des Root-Benutzers |
| `/sbin` | System-Programme für root |
| `/srv` | Daten für Dienste (Webserver, FTP, ...) |
| `/sys` | Treiber-/Kernel-Informationen |
| `/tmp` | Temporäre Dateien |
| `/usr` | Softwarepakete des Herstellers |
| `/usr/local` | Lokal installierte Software |
| `/var` | Variable Daten (Logs, Emails, ...) |

## Wildcards (Pattern-Matching)

| | |
|---|---|
| `*` | Beliebiger String (auch leer) |
| `?` | Genau ein beliebiges Zeichen |
| `[a-zA-Z]` | Genau eines der angegebenen Zeichen(bereiche) |

```
# Verzeichnisinhalt: a  abcd  a.txt  b  bc  d  d3
*.txt      # -> a.txt
*b?        # -> bc
*d*        # -> abcd d d3
[a-z][0-9] # -> d3
```

## Textdateien anzeigen und bearbeiten

| | |
|---|---|
| `cat <file>` | Datei komplett ausgeben |
| `less <file>` | Datei seitenweise anzeigen |
| `head [-n x] <file>` | Erste x Zeilen anzeigen (Standard: 10) |
| `tail [-fn x] <file>` | Letzte x Zeilen (`-f`: follow, ideal für Logs) |
| `nano <file>` | Texteditor im Terminal |
| `vi(m) <file>` | Mächtiger Texteditor |
| `wc <file>` | Zeilen, Wörter, Zeichen zählen |

```
tail -f /var/log/syslog     # Log-Datei live beobachten
head -n 5 datei.txt         # Erste 5 Zeilen anzeigen
wc report.txt               # 438  2115  18302  report.txt
```

## Suchen: find und grep

```
# find: Dateien und Verzeichnisse suchen
find . -size +500c                          # Dateien > 500 Zeichen
find . -name "*.txt" -a -mtime -1           # .txt geaendert in < 24h
find . -type d                              # Alle Unterverzeichnisse
find ~ ! -type d                            # Alle Nichtverzeichnisse im Home

# grep: In Dateien suchen
grep -i linux *.txt        # Gross-/Kleinschreibung ignorieren
grep -r Linux .            # Rekursiv in allen Unterverzeichnissen
grep -n "Fehler" log.txt   # Zeilennummern anzeigen
find . -name "*.txt" -exec grep Linux {} \;
```

## Benutzer und Rechte

| | |
|---|---|
| `w, who` | Wer ist angemeldet? |
| `whoami` | Wer bin ich? |
| `groups` | Meine Gruppen |
| `id` | Meine UserID |
| `passwd` | Passwort ändern |
| `su <user>` | Benutzer wechseln |
| `sudo <cmd>` | Befehl als root ausführen |

### Dateirechte

| | |
|---|---|
| `chmod ugoa[+-=]rwx <file>` | Rechte symbolisch ändern |
| `chmod 755 <file>` | Rechte oktal ändern (rwxr-xr-x) |
| `chown user:group <file>` | Besitzer/Gruppe ändern |

| Oktal | Binär | Kürzel | Bedeutung |
|---|---|---|---|
| 0 | 000 | `--` | nichts |
| 1 | 001 | `-x` | ausführen |
| 4 | 100 | `r-` | lesen |
| 5 | 101 | `r-x` | lesen, ausführen |
| 6 | 110 | `rw-` | lesen, schreiben |
| 7 | 111 | `rwx` | lesen, schreiben, ausführen |

```
chmod ug+rwx test           # Volle Rechte fuer User+Gruppe
chmod og-rwx ~/Documents    # Vor allen anderen verstecken
chmod a+rx ~/bin/prog       # Fuer alle ausfuehrbar
chmod -R 777 *              # Alles rekursiv voll berechtigen (unsicher!)
```

## Prozesse

| | |
|---|---|
| `ps` | Laufende Prozesse anzeigen |
| `ps axww` | Alle Prozesse detailliert |
| `top` | Fortlaufende Anzeige der aktivsten Prozesse |
| `bg` | Gestoppten Prozess in den Hintergrund |
| `fg` | Prozess wieder in den Vordergrund |
| `kill <pid>` | Prozess beenden |
| `killall <name>` | Prozesse mit diesem Namen beenden |
| `<cmd> &` | Befehl im Hintergrund starten |

## Ein-/Ausgabe-Umleitung

| | |
|---|---|
| `> <file>` | stdout in neue Datei umleiten |
| `» <file>` | stdout an Datei anhängen |
| `>&` | stderr & stdout umleiten |
| `< <file>` | Datei als stdin lesen |
| `/dev/null` | Ausgabe verschlucken |
| `/dev/zero` | Leere Eingabe |

```
grep Hase *.txt > hasen          # Treffer in Datei "hasen"
grep Igel *.txt >> hasen         # Treffer anhaengen
ls *.txt >& errors > txt         # Fehler in "errors", .txt in "txt"
./prog < /dev/zero >& stderr > /dev/null &  # Im Hintergrund ohne I/O
```

## Pipes und Verknüpfungen

| | |
|---|---|
| `cmd1 \| cmd2` | Ausgabe von cmd1 als Eingabe von cmd2 |
| `cmd1 ; cmd2` | cmd2 nach cmd1 starten |
| `cmd1 && cmd2` | cmd2 nur bei Erfolg von cmd1 |
| `cmd1 \|\| cmd2` | cmd2 nur bei Fehler von cmd1 |
| `$?` | Rückgabewert des letzten Befehls |

```
ps axww | grep bash | grep -v grep  # Alle bash-Shells finden
ls -a | grep txt                     # Dateien mit "txt" im Namen
ls *.txt || echo "Keine .txt hier"
test -f "bla" && cat bla             # Datei nur ausgeben wenn existent
cd bla; ls                           # Wechseln und auflisten
```

## Shell-Variablen

```
name=bla; echo "name ist $name"      # Setzen und Auslesen
i="Meine Fotos"                      # Leerzeichen -> Anfuehrungszeichen!
# FALSCH: i=Meine Fotos   (Leerzeichen trennt -> Fehler)
# FALSCH: i = 1            (Leerzeichen um = -> Fehler)

# Skript-Parameter
$0            # Skriptname
$1, $2, ...   # Parameter 1, 2, ...
$*            # Alle Parameter
$?            # Rueckgabewert des letzten Befehls

# Backtick: Befehlsausgabe als String
files=`find . -name "*.txt"`; echo $files
datum=`date +%Y%m%d`
```

### Musterersetzung in Variablen

```
TEST=/home/hotz/abc.txt.old
${TEST%.old}        # /home/hotz/abc.txt    (Ende entfernen)
${TEST%.*}          # /home/hotz/abc.txt    (letzter Punkt ab)
${TEST%%.*}         # /home/hotz/abc        (erster Punkt ab)
${TEST#*/}          # home/hotz/abc.txt.old (erstes / entfernen)
${TEST##*/}         # abc.txt.old            (letztes / entfernen)
${TEST//ab/de}      # /home/hotz/dec.txt.old (alle ab->de)
```

## Umgebungsvariablen

| | |
|---|---|
| `export VAR=val` | Variable exportieren (für Kindprozesse) |
| `unset VAR` | Variable löschen |
| `PWD` | Aktuelles Verzeichnis |
| `HOME` | Pfad zum Home |
| `PATH` | Befehlssuchpfad (durch `:` getrennt) |
| `DISPLAY` | X-Server (meist `:0.0`) |
| `PS1` | Shell-Prompt |
| `EDITOR` | Standard-Texteditor |
| `LANG` | Spracheinstellung |

```
PATH="~/bin:$PATH"; echo $PATH
export EDITOR=nano
```

## Aliases

```
alias cp='cp -i' mv='mv -i' rm='rm -i'  # Nachfrage vor Ueberschreiben
alias ls='ls --color=auto'
alias ll='ls -l'
unalias ll                # Alias loeschen
alias                     # Alle Aliases anzeigen
```

## Skripte

```
#!/bin/bash
# Shebang in Zeile 1 bestimmt ausfuehrende Shell
mkdir -p $2 && chmod og-rwx $2 && mv $1 $2
# Ausfuehrbar machen und starten:
# chmod a+rx mvtonewdir.sh
# ./mvtonewdir.sh bla test
```

## Schleifen

### for-Schleife

```
for var in wort1 wort2 wort3; do
  echo $var
done

for f in *.txt; do
  n=${f%.txt}-2.txt; mv $f $n   # Alle .txt umbenennen
done

for f in 1 2 3 4; do echo -n "$f,"; done   # 1,2,3,4,
for var in "$@"; do echo $var; done         # Alle Skript-Parameter
for datei in *; do echo $datei; done        # Alle Dateien
```

### while-Schleife

```
COUNTER=0
while [ $COUNTER -lt 10 ]; do
  echo "Counter ist $COUNTER"
  let COUNTER=COUNTER+1       # Alternative: ((COUNTER++))
done

# Endlos-Einzeiler:
while true; do [Befehl]; sleep 1; done
```

### until-Schleife

```
COUNTER=20
until [ $COUNTER -lt 10 ]; do
  echo "COUNTER $COUNTER"
  let COUNTER-=1
done
```

## Bedingte Ausführung – if / test

```
if test -f $f; then echo "$f gibt es"; fi
if [ -f $f ]; then echo "$f gibt es"; fi     # [] = test
if ! test -f $f; then echo "$f fehlt"; fi
if grep -q bla $f; then echo "bla in $f"; fi
```

### Vergleichsoperatoren (test)

| | |
|---|---|
| ßtr1- ßtr2" | Strings gleich |
| ßtr1"!= ßtr2" | Strings ungleich |
| -eq, -ne | Zahlen gleich / ungleich |
| -gt, -ge | Zahlen größer (gleich) |
| -lt, -le | Zahlen kleiner (gleich) |
| -f "datei" | Ist reguläre Datei |
| -d "datei" | Ist Verzeichnis |
| -o / -a | Logisches ODER / UND |

## Shellexpansion

```
~/Documents            # -> /home/hotz/Documents (Tildeexpansion)
echo "Host: $HOSTNAME"  # -> Host: raspi (Variablenexpansion)
echo 'Host: $HOSTNAME'  # -> Host: $HOSTNAME (keine Expansion in '')
echo `date +%Y%m%d`     # -> 20190309 (Backtick-Ersetzung)
```

## sed – Stream Editor

```
# Suchen und Ersetzen
sed 's/alt/NEU/' inputfile          # Ersten Treffer ersetzen
sed 's/alt/NEU/g' inputfile         # Alle Treffer ersetzen
sed -i 's/alt/NEU/g' datei          # In-Place (Datei direkt aendern)

# Wichtige sed-Parameter
sed -n '1,5p' datei        # Nur Zeilen 1-5 anzeigen
sed '4d' datei              # Vierte Zeile loeschen
sed '1a\ "Neue Zeile"' d   # Text nach Zeile 1 einfuegen
sed '2r neu.txt' datei      # Inhalt von neu.txt nach Zeile 2

# Gruppierung mit \( \)
echo "eins zwei drei vier" | sed 's/[a-z]* [a-z]* \([a-z]*\) .*/\1/'
# -> drei

# Praxisbeispiel: Temperatur extrahieren
cat .../w1_slave | sed -n '2p' | sed 's/.* t=\([0-9]*\)\([0-9]\{3\}\)/\1.\2/'
cat .../w1_slave | grep t | sed 's/.*=//'
```

### Reguläre Ausdrücke (RegExp)

| | |
|---|---|
| . | Ein beliebiges Zeichen |
| [abc] | Eines der Zeichen a, b, c |
| [^abc] | Beliebiges Zeichen außer a, b, c |
| [a-z] | Zeichenbereich a bis z |
| * | 0 oder mehr Wiederholungen |
| + | 1 oder mehr Wiederholungen |
| ? | 0 oder 1 Wiederholung |
| {n} | Genau n Wiederholungen |
| {n,m} | Zwischen n und m Wiederholungen |

## awk und cut

```
# awk: Zeilenweise auswerten
awk '$2 == "localhost" { print $1 }' /etc/hosts
cat /proc/cpuinfo | grep Serial | awk '{print $3}'
cat /etc/passwd | grep pi | awk -F : '{ print $1 $3 }'

# cut: Spalten extrahieren
cut -c 18-25 /proc/meminfo            # Zeichen 18-25 jeder Zeile
cat /proc/cpuinfo | grep Serial | cut -d : -f 2
cat /etc/passwd | grep pi | cut -d : -f 3   # UID von User pi
```

## Nützliche Tools

| | |
|---|---|
| `date` | Datum/Uhrzeit anzeigen |
| `sleep <n>` | n Sekunden warten |
| `sort [-rnu] <file>` | Zeilen sortieren (`-r` umgekehrt, `-n` numerisch, `-u` unique) |
| `tee [-a] <file>` | Ausgabe gleichzeitig auf Bildschirm und in Datei |
| `bc -l` | Bash-Berechnungen (`bc -l <<< 5*5`) |
| `df -h [dir]` | Plattenbelegung anzeigen |
| `ldd <prog>` | Benötigte Bibliotheken anzeigen |

```
# tar: Archivieren
tar czf archiv.tar.gz verzeichnis/   # Erzeugen (gzip)
tar xzf archiv.tar.gz                # Entpacken (gzip)
tar cjf archiv.tar.bz2 verzeichnis/  # Erzeugen (bzip2)

# wget: Dateien herunterladen
wget -c https://example.com/file.iso     # Abgebrochenen Download fortsetzen
wget -m https://www.heise.de/            # Seite spiegeln

# make: Ausgabe loggen
make | tee build.log
make install | tee -a build.log
```

## Netzwerk

| | |
|---|---|
| `ssh [-X] [user@]host` | Terminal auf anderem Rechner (`-X`: X11-Forwarding) |
| `scp <src> <host>:<dst>` | Datei per SSH kopieren |
| `nc <host> <port>` | Verbindung zu einem Port aufbauen |
| `nc -l -p <port>` | Als Dienst auf Port lauschen |
| `hostname` | Rechnername anzeigen |
| `netstat` | Aktive Ports anzeigen |
| `ifconfig` | Netzwerkkarten-Infos (IP, MAC) |

Wichtige Ports: `http=80`, `ssh=22`. IPv4: `a.b.c.d` mit $0 \leq a, b, c, d \leq 255$. Localhost = `127.0.0.1`.

## Konfigurationsdateien

| | |
|---|---|
| `~/.bashrc, ~/.profile` | Shell-Start/Login: Variablen, Aliases (`source .bashrc` zum Testen!) |
| `~/.ssh/config` | SSH-Einstellungen (z.B. `ForwardX11 Yes`) |
| `~/.vimrc` | vim-Einstellungen (z.B. `syntax on`) |
| `/etc/shadow` | Passwort-Hashes der Benutzer |

## Virtual Terminals: screen / tmux

| screen | | tmux | |
|---|---|---|---|
| `screen -x` | Sitzung fortsetzen | `tmux a` | Sitzung fortsetzen |
| `Ctrl-a c` | Neues Fenster | `Ctrl-b c` | Neues Fenster |
| `Ctrl-a n/p` | Nächstes/Voriges | `Ctrl-b n/p` | Nächstes/Voriges |
| `Ctrl-a d` | Sitzung trennen | `Ctrl-b d` | Sitzung trennen |

## GPIO steuern – pinctrl (ab Kernel 6.6)

```
pinctrl get 10          # Zustand an GPIO10 anzeigen
pinctrl get 10,11       # Mehrere GPIOs gleichzeitig
pinctrl set 16 op       # GPIO16 als Ausgang konfigurieren
pinctrl set 16 dh       # GPIO16 auf HIGH (3,3V)
pinctrl set 16 dl       # GPIO16 auf LOW  (0V)
# Optionen: ip=Input, op=Output, pu=Pull-Up, pd=Pull-Down, pn=No-Pull
```

### GPIO steuern – sysfs (vor Kernel 6.6)

```
echo "16" > /sys/class/gpio/export            # GPIO16 aktivieren
echo "out" > /sys/class/gpio/gpio16/direction  # Als Ausgang
echo "1" > /sys/class/gpio/gpio16/value        # HIGH setzen
```

## Sensoren auslesen (Bash)

### DS18B20 Temperatursensor (1-wire)

```
# Temperatur auslesen (nur Ganzzahl)
cat /sys/bus/w1/devices/28-.../w1_slave | tail -n 1 | cut -d= -f2
cat /sys/bus/w1/devices/28-.../w1_slave | grep t | sed 's/.*=//'
```

```
# Mit Dezimalpunkt per sed
cat .../w1_slave | sed -n '2p' \
  | sed 's/.* t=\([0-9]*\)\([0-9]\{3\}\)/Temperatur: \1.\2C/'
```

### I2C-Bus

```
sudo apt install i2c-tools
i2cdetect -y 1        # Bus scannen, Slave-Adressen finden
i2cdump ...            # Daten lesen
i2cget ...             # Einzelwert lesen
i2cset ...             # Daten an Slave schreiben
```

## RRDtool – Round-Robin-Database

```
sudo apt install rrdtool

# Datenbank erzeugen
rrdtool create temperatur1.rrd --step 5 \
  DS:temp1:GAUGE:600:-30:100 \
  RRA:AVERAGE:0.5:1:17280

# Daten einfuegen (N = jetzt)
rrdtool update temperatur1.rrd N:$value

# Graph erzeugen
rrdtool graph temp.png --start -1hour --end now \
  --width 640 --height 100 --vertical-label "Grad Celsius" \
  DEF:temp1=temperatur1.rrd:temp1:AVERAGE \
  LINE1:temp1#0000FF:'Temperatur'
```

## Webserver (Lighttpd)

```
sudo apt install lighttpd
# Webseiten liegen unter: /var/www/html/

# CGI aktivieren
sudo lighty-enable-mod cgi
sudo nano /etc/lighttpd/conf-available/10-cgi.conf
sudo mkdir /var/www/html/cgi-bin
sudo chown www-data:www-data /var/www/html/cgi-bin
sudo service lighttpd restart
```

## Crontab / Autostart

```
crontab -e                              # Crontab bearbeiten
@reboot /home/hotz/autostartup.sh       # Skript beim Boot ausfuehren

# Autostart-Skript Beispiel:
#!/bin/bash
tmux new -s "RRD Update Skripte" -d
tmux set-option -g default-shell "/bin/bash"
tmux new-window "/home/hotz/updatetemp1.sh"
```

## GIT

```
git init                                # Neues Repository
git clone https://github.com/.../repo.git  # Auschecken
git add <datei>                         # Aenderungen vorschlagen
git add *                               # Alle Aenderungen
git commit -m "Nachricht"               # Aenderungen bestaetigen
git push origin master                  # Zum Remote senden
git pull                                # Vom Remote aktualisieren
git config --global credential.helper store  # Passwort speichern
```

## Passwort-Hashes (/etc/shadow)

| | |
|---|---|
| $1$ | MD5 |
| $2a$ | Blowfish |
| $5$ | SHA-256 |
| $6$ | SHA-512 |
| $y$ | yescrypt |

Format: `user:$hash:Tage_seit_1970:min:max:warn:inaktiv:deaktiviert:reserviert`

# Teil 2 – Python

## Grundlagen

```
#!/usr/bin/python3
print("Hello World")          # Ausgabe
help(print)                   # Hilfe
exit()                        # oder Ctrl-d
```

## Datentypen

```
# Ganzzahlen
print(42)             # 42
print(42/3)           # 14.0  (Division liefert float!)
print(42 // 3)        # 14    (Ganzzahldivision)

# Fliesskommazahlen
print(6.023e23)       # 6.023e+23

# Zeichenketten
print("Hello")        # Doppelte Anfuehrungszeichen
print('Hello')        # Einfache Anfuehrungszeichen
print("""Ueber
mehrere Zeilen""")    # Dreifache Anfuehrungszeichen

int("42")             # String -> Ganzzahl
float("3.14")         # String -> Fliesskomma
str(42)               # Zahl -> String
```

## Variablen und Operatoren

```
var1 = 2
var2 = 4
var1 = var1 * var2     # var1 ist jetzt 8
```

| | |
|---|---|
| `+` | Addition, bei Strings verketten: `ä"+"b"` → `äb"` |
| `-` | Subtraktion |
| `*` | Multiplikation, Strings vervielfältigen: `äb*2` → `äbab"` |
| `/` | Division (liefert float) |
| `//` | Ganzzahldivision |
| `%` | Modulo (Rest) |
| `**` | Exponent: `3**2` → `9` |

### Bitweise Operatoren

| | |
|---|---|
| `&` | AND |
| `\|` | OR |
| `^` | XOR |
| `~` | NOT (alle Bits invertieren) |
| `«` | Links-Shift (`x « y = x * 2**y`) |
| `»` | Rechts-Shift (`x » y = x / 2**y`) |

## Listen

```
kaufen = ["Muesli", "Milch", "Obst"]
kaufen[1] = "Sahne"           # Element aendern
print(kaufen[0])              # Muesli
print(kaufen[-1])             # Obst (letztes Element)
print("Saft" in kaufen)      # False

kaufen.append("Brot")        # Am Ende anfuegen
kaufen.insert(1, "Saft")     # An Position 1 einfuegen
kaufen.remove("Milch")       # Erstes Vorkommen entfernen
del kaufen[-1]                # Element per Index loeschen

# Unterlisten (Slicing)
print(kaufen[1:3])            # Elemente 1 bis 2
print(len(kaufen))            # Laenge

# Sortieren
A = [2,1,3,5,4]
B = sorted(A)                # A bleibt gleich
A.sort()                      # A wird sortiert
```

```
# Listen aus Listen (List Comprehension)
[a**2 for a in range(5)]                # [0, 1, 4, 9, 16]
[a for a in range(10) if a % 2 == 1]    # [1, 3, 5, 7, 9]

# ACHTUNG: Flache vs. echte Kopie
merken = kaufen       # Flache Kopie (Referenz!)
merken = kaufen[:]    # Echte Kopie (Deep Copy)
```

## Tupel

```
t = ("Muesli", "Kaese", "Milch")    # Unveraenderbare Liste
# t[1] = "X"  -> TypeError!
A, B = B, A                          # Werte tauschen per Tupel
```

## Wörterbücher (Dicts)

```
de_en = {"Milch": "milk", "Mehl": "flour"}
de_en["Eier"] = "eggs"            # Neues Paar hinzufuegen
print(de_en["Milch"])             # milk
if "Mehl" in de_en: print("ok")
for key, val in de_en.items():    # Ueber alle Paare iterieren
    print(key, "=>", val)
```

## Stringmethoden

```
"Hallo Welt".find("Welt")          # 6 (-1 wenn nicht gefunden)
"abcabc".replace("abc", "123")     # "123123"
"hallo".capitalize()               # "Hallo"
"Hallo".upper()                    # "HALLO"
"HALLO".lower()                    # "hallo"
"1,2,3".split(",")                 # ['1', '2', '3']
"  Hallo  ".strip()                # "Hallo"
```

## Formatierte Ausgabe

```
# .format() Methode (empfohlen)
print("{}^2 + {}^2 = {:05}^2".format(3, 4, 5))
# -> 3^2 + 4^2 = 00005^2

print("Strings {1} {0:>10}".format("Welt", "Hallo"))
# -> Strings Hallo       Welt

print("Fliess {:e} {:+8.4f} {:g}".format(3.14, 3.14, 3.14))
# -> Fliess 3.140000e+00   +3.1400 3.14

# Alte Methode (noch funktionsfaehig)
print("Nr: %5d, Preis: %8.2f" % (123, 19.589))
```

## Kontrollstrukturen

### if / elif / else

```
a = 1
if a < 5:
    print("a ist kleiner als 5")
elif a > 5:
    print("a ist groesser als 5")
else:
    print("a ist 5")
```

### Logische Operatoren

| | |
|---|---|
| `==, !=` | Gleichheit / Ungleichheit |
| `<, >, <=, >=` | Vergleiche |
| `and, or, not` | Logische Verknüpfungen |

### for-Schleife

```
for language in ["C", "C++", "Python"]:
    print(language)

# range(start, stop, step)
range(10)         # [0, 1, ..., 9]
range(3, 12)      # [3, 4, ..., 11]
range(5, 50, 5)   # [5, 10, 15, ..., 45]
range(10, 0, -1)  # [10, 9, ..., 1]
```

```
for zahl in range(10):
    if zahl == 3:
        break       # Schleife sofort verlassen
        continue    # Nur diesen Durchlauf ueberspringen
else:
    print("Kein break")  # Nur wenn KEIN break ausgefuehrt wurde
```

### while-Schleife

```
a = 1
while a < 5:
    a += 1
```

## Funktionen

```
def printMax(a, b):
    """Gibt das Maximum von a und b aus."""
    if a > b: print(a)
    else: print(b)

def lj(r, epsilon=1.0, sigma=1.0):   # Vorgabewerte
    return 4*epsilon*((sigma/r)**6 - (sigma/r)**12)

lj(1.0, sigma=0.5)    # Argumente benennen -> Reihenfolge egal
```

**Wichtig:** Variablen in Funktionen sind *lokal*. Globale Variablen können nur gelesen werden. `pass` für leere Blöcke. Einrückung definiert Blöcke!

## Module

```
import sys                      # sys-Modul laden
from random import random       # Einzelne Funktion importieren
help(modul)                     # Hilfe
dir(modul)                      # Alle Funktionen anzeigen
```

### sys-Modul

```
import sys
sys.argv[0]           # Programmname
sys.argv[1]           # Erster Parameter
sys.stdin.readline() # Zeile von stdin lesen
sys.stderr.write("Fehler\n")
```

### os-Modul

```
import os, os.path
os.path.dirname(file)     # Verzeichnis
os.path.basename(file)    # Dateiname
os.path.join(dir, name)   # Pfade zusammenfuegen
os.path.exists(path)      # Existiert?
os.path.isdir(path)       # Ist Verzeichnis?
os.mkdir(dir)             # Verzeichnis erstellen
os.rename(old, new)       # Umbenennen
os.remove(file)           # Datei loeschen
```

### subprocess-Modul

```
import subprocess
subprocess.run(["ls", "-l"])
subprocess.run(["ls -l"], shell=True)
output = subprocess.check_output("cat list.txt | awk '{print $2}'",
                                 shell=True)
```

### argparse-Modul

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("-f", "--file", dest="filename", help="FILE")
parser.add_argument("number", type=int, help="die Zahl")
args = parser.parse_args()
# python prog.py -f test 1 -> args.filename='test', args.number=1
```

### math / random

```
import math, random
math.sqrt(2)              # Wurzel
math.pi                   # Pi
```

```
math.log(x)                # Logarithmus
random.random()            # Zufallszahl [0, 1)
random.randint(1, 10)      # Ganzzahl [1, 10]
random.gauss(0, 1)         # Normalverteilt
```

## Datei-Ein-/Ausgabe

```
# Lesen
f = open("in.txt")           # oder open("in.txt", "r")
line = f.readline()           # Eine Zeile
text = f.read()               # Gesamte Datei
for line in f:                # Datei als Sequenz
    print(line)

# Schreiben
f = open("out.txt", "w")     # w=neu, a=anhaengen
f.write("Hallo\n")           # Kein auto. Zeilenumbruch!
f.close()                     # Wichtig: Daten werden erst beim close geschrieben
```

## GPIO mit Python (RPi.GPIO)

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)           # BCM-Nummerierung
GPIO.setup(17, GPIO.OUT)         # GPIO17 als Ausgang
GPIO.output(17, True)            # GPIO17 HIGH
GPIO.output(17, False)           # GPIO17 LOW
GPIO.setup(18, GPIO.IN)          # GPIO18 als Eingang
val = GPIO.input(18)             # GPIO18 lesen
```

## SPI mit Python

```
import spidev
spi = spidev.SpiDev()
spi.open(0, 0)                   # Bus 0, CS0
spi.readbytes(4)                 # 4 Bytes lesen
spi.writebytes([0x01, 0x02])     # Bytes senden
spi.xfer([0x80, 0xF8], 6400000) # Senden+Empfangen, Freq
spi.close()
```

## I2C mit Python (smbus)

```
import smbus
bus = smbus.SMBus(1)                        # I2C-Bus 1
data = bus.read_i2c_block_data(0x77, 0x88, 24)  # 24 Bytes ab Register 0x88
bus.write_byte_data(0x77, 0xF4, 0x27)       # Byte an Register schreiben
```

## WS2812b RGB-LEDs (SPI)

```
def write2812(spi, color):
    tx = []
    for rgb in color:
        for byte in [rgb[1], rgb[0], rgb[2]]:  # GRB-Reihenfolge!
            for ibit in range(7, -1, -1):
                tx.append(0x80 if (byte >> ibit) & 1 == 0 else 0xF8)
    spi.xfer(tx, int(8 / 1.25e-6))

import spidev
spi = spidev.SpiDev()
spi.open(0, 0)
nLED = 8
write2812(spi, [[0, 5, 25]] * nLED)      # 8 LEDs: R=0, G=5, B=25
```

## InfluxDB mit Python

```
from influxdb import InfluxDBClient
import time

client = InfluxDBClient('127.0.0.1', '8086', 'admin', 'geheim', 'raspi')

data = [{
    "measurement": "umwelt",
    "time": time.asctime(time.gmtime()),
    "fields": {
        "Temperatur": temperature,
        "Luftdruck": pressure
    }
}]
```

```
client.write_points(data, time_precision='s')
```

## RRDtool mit Python

```
import rrdtool
# sudo apt install librrd-dev python3-rrdtool

rrdtool.create("temp.rrd", "--step", "5",
    "DS:temp1:GAUGE:600:-30:100",
    "RRA:AVERAGE:0.5:1:17280")

rrdtool.update("temp.rrd", "N:{}".format(value))

rrdtool.graph("temp.png",
    "--start", "-1hour", "--end", "now",
    "--width", "640", "--height", "100",
    "DEF:t=temp.rrd:temp1:AVERAGE",
    "LINE1:t#0000FF:Temperatur")
```

## JSON Format

```
# JSON-Datentypen: null, true/false, Zahl, String, Array [], Objekt {}
# Beispiel:
data = {
    "firstName": "Jane",
    "age": 35,
    "hobbies": ["running", "singing"],
    "children": [
        {"firstName": "Alice", "age": 6},
        {"firstName": "Bob", "age": 8}
    ]
}
```

## Raspberry Pi Kamera

```
# raspi-config -> Kamera aktivieren
raspistill -o bild1.jpg              # Bild aufnehmen
raspivid -t 100000 -o video1.h264   # Video (t in ms)

# Motion (Ueberwachungskamera)
sudo apt install motion
sudo modprobe bcm2835-v4l2          # Kernel-Modul laden
# Konfiguration: /etc/motion/motion.conf
# Livestream: http://[raspi-ip]:8081
```

## Grafana

```
sudo apt install grafana
sudo systemctl enable grafana-server
sudo systemctl start grafana-server
# Web-UI: http://<ip>:3000  (admin/admin)
```

*Viel Erfolg bei der Klausur!*